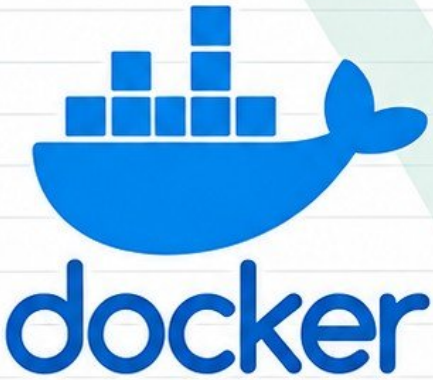
VERIQTA

7 Docker Compose Setups You'll Reuse Forever



+

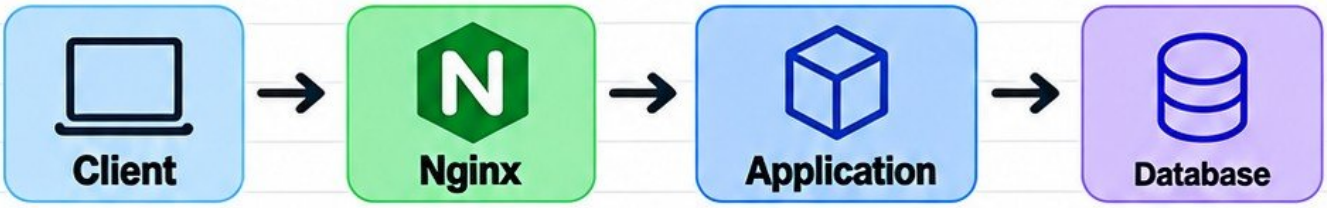


Docker Compose

- Real-world setups
- Practical examples
- Troubleshooting guides
- Security best practices
- Production-ready configs

compose.yaml

```yaml
version: '3.8'
services:
  web:
    image: nginx:latest
    ports:
      - "80:80"
  app:
    build: ./app
    environment:
      - DB_HOST=db
```



Build • Deploy • Manage • Troubleshoot • Reuse

Why Docker Compose Exists

The Problem

- Real applications usually need multiple services (e.g., a web app, database, and cache).
- Running and managing each container separately becomes complex, error-prone, and hard to reproduce.

Docker Engine vs Docker Compose

Docker Engine	■ Runs and manages individual containers. ■ You start, stop, and connect containers one by one.
Docker Compose	■ A tool for defining and running multiple containers together. ■ Uses one YAML file to describe the entire application stack.

How It Works

- You write a YAML file (e.g., compose.yaml) that lists your services (e.g., web app, database), their configuration, networks, and volumes.
- Then you start the whole stack with one command (e.g., docker compose up -d).

Example: Web App + Database

```yaml
services:
  web:
    image: nginx:latest
    ports:
      - "8080:80"
  db:
    image: postgres:16
    environment:
      POSTGRES_DB: appdb
```

Web Application Service

Database Service

Application Flow



Junior Engineer Insight: One container is not necessarily one complete application.

Anatomy of a Real Compose File

Key Compose Elements

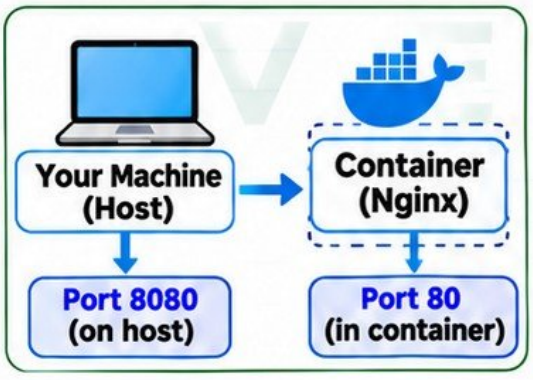
- **services:** A list of containers that make up your application.
- **image:** The container image to use (e.g., nginx:latest).
- **build:** Build an image from a Dockerfile.
- **ports:** Map a port on your host to a port in the container.
- **environment:** Set environment variables for the container.
- **volumes:** Persist data outside the container.
- **networks:** Define a private network for your services.

compose.yaml (Minimal Example)

```yaml
services:
  web:
    image: nginx:latest
    ports:
      - "8080:80"
    environment:
      - NGINX_HOST=localhost
    volumes:
      - web-data:/usr/share/nginx/html
  db:
    image: postgres:16
    environment:
      POSTGRES_DB: appdb
      POSTGRES_USER: appuser
      POSTGRES_PASSWORD: secret123

volumes:
  web-data:
```

Ports: Host vs Container



- **"8080:80"** means:
  - Access the app on port 8080 on your machine.
  - It is forwarded to port 80 inside the container.

Validate Your Configuration

$ docker compose config

- Checks your compose file for errors.
- Shows the final, resolved configuration.
- Use it before running the stack.

Common Mistake

Confusing YAML indentation (spacing) with application behavior. Wrong indentation can cause services to be misconfigured or fail to start.

The Docker Compose Command Workflow

What You'll Learn

- The key Docker Compose commands.
- What each command does and how it changes your environment.
- A simple workflow to start, inspect, view logs, and stop your stack safely.

Command Workflow at a Glance



compose.yaml (Defines your stack) → (Runs and manages the stack) → Running Containers (Your services)

Key Docker Compose Commands

COMMAND	WHAT IT DOES	EXAMPLE
docker compose up -d	Builds (if needed) and starts all services in the background (detached mode).	$ docker compose up -d
docker compose ps	Shows the status of all services (containers) in your stack.	$ docker compose ps
docker compose logs -f	Streams real-time logs from your services (follow mode).	$ docker compose logs -f
docker compose exec	Runs a command inside a running service container (e.g., shell access).	$ docker compose exec web bash
docker compose down	Stops and removes the containers, network, and default resources.	$ docker compose down

Practice: Try This

1. Start your stack: docker compose up -d
2. Inspect running services: docker compose ps
3. View logs: docker compose logs -f
4. Access a container: docker compose exec web bash
5. Stop the stack: docker compose down

Warning

docker compose down -v also removes volumes, which **can delete your data** (e.g., database files, uploads). Use it only when you are sure you don't need the data.

Nginx Reverse Proxy + Web Application

What You'll Build

- A simple stack with Nginx as a reverse proxy in front of a web application.
- Nginx receives HTTP traffic and routes itto the app using the service name (e.g., app) on the internal Docker network.

Architecture



docker-compose.yaml (Example)

```yaml
services:
  nginx:
    image: nginx:latest
    ports:
      - "80:80"
    volumes:
      - ./nginx/default.conf:/etc/nginx/
        conf.d/default.conf
    depends_on:
      - app
  app:
    image: node:20-alpine
    working_dir: /app
    volumes:
      - ./app:/app
    command: npm start
    expose:
      - "3000"
```

Key Concepts

- **Reverse Proxy:** Nginx sits in front of the app and forwards requests to it.
- **Service Name:** Use the Compose service name (app) instead of an IP address.
- **Ports:** "80:80" exposes port 80 on your host; expose: "3000" keeps port 3000 internal.
- **Network:** Compose creates a private network so services can talk by name.

Test It

```
docker compose up -d      # start the stack
docker compose ps         # check services
curl http://localhost     # should return the app
```

Troubleshoot: 502 Bad Gateway

What it means: Nginx cannot reach the upstream application.

Common causes:

- The app container is not running.
- Wrong service name in Nginx config.
- App is running on a different port.
- The app is starting slowly.

How to Fix

- Check container status: `docker compose ps`
- Check app logs: `docker compose logs app`
- Verify Nginx upstream: `proxy_pass http://app:3000;`
- Ensure the app is listening on port 3000.
- Restart if needed: `docker compose restart app`

Junior Engineer Insight:

A reverse proxy improves security, simplifies access, and lets you run multiple apps behind one port.

Make the Reverse Proxy Reusable

1. Mount an Nginx Configuration File

- Keep Nginx settings in a file and mount it into the container so you can reuse and update it without rebuilding the image.

compose.yaml (snippet)

```
services:
  nginx:
    image: nginx:latest
    ports:
      - "80:80"
    volumes:
      - ./nginx/nginx.conf:/etc/nginx/
        nginx.conf:ro
```

2. Ports vs Expose

Feature	ports	expose
Purpose	Publishes a container port to the host (external access).	Makes a port available only to other containers.
Example	`ports:` `- "80:80"`	`expose:` `- "3000"`
Use case	Access from your browser or outside the host.	Internal communication between services.

3. Inspect Logs

- Check logs to see what Nginx and your application are doing.

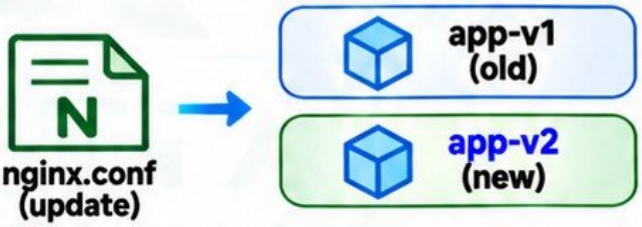
```
docker compose logs nginx
# View Nginx (proxy) logs

docker compose logs app
# View application logs

docker compose logs -f
# Follow logs in real time
```

4. Practice: Change the Upstream

- Update the Nginx config to point to a different service (e.g., app-v2) and verify that traffic is routed correctly.




```
curl http://localhost   # verify routing
```

5. Production Connection

TLS Termination

- Use Nginx to handle HTTPS (TLS) with your domain certificate.
- Keeps requests encrypted between users and your stack.

Health Checks

- Add a health check to ensure the application is ready and healthy.
- Example: check "/health" endpoint before routing.

Limit Public Exposure

- Expose only what is needed (e.g., port 80/443).
- Keep database and other internal services private (using expose and networks).

Junior Engineer Insight: A reusable reverse proxy setup makes it easy to add new apps, switch versions, and run them securely.

PostgreSQL + Persistent Storage

What You'll Do

- Deploy PostgreSQL using a Compose service.
- Configure database name, username, and password.
- Use a named volume for database persistence.
- Connect to the database using psql.
- Understand why deleting a container should not erase your data.

Architecture



Key Concepts

POSTGRES_DB: The database name to create.

POSTGRES_USER: The database username.

POSTGRES_PASSWORD: The user's password.

Named Volume: Stores database data outside the container.

compose.yaml (PostgreSQL)

```
services:
  postgres:
    image: postgres:16
    environment:
      POSTGRES_DB: appdb
      POSTGRES_USER: appuser
      POSTGRES_PASSWORD: secret123
    ports:
      - "5432:5432"
    volumes:
      - pgdata:/var/lib/postgresql/data
volumes:
  pgdata:
```

Connect Using psql

```
docker compose up -d          # start PostgreSQL
docker compose ps             # verify it's running
docker compose exec postgres psql -U appuser -d appdb
appdb=# SELECT version();
PostgreSQL 16.x ...
```

Persistence: What It Means



Why Data Isn't Lost

- The container is temporary (can be deleted).
- The database files live in a named volume (pgdata) on your host.
- When you recreate the container, it reuses the same volume and data.

Junior Engineer Insight: A container is just a runtime. Your data must live in a persistent volume so it survives container deletion.

Database Backups and Recovery

What You'll Do

- Use pg_dump for logical backups.
- Restore a database using pg_restore where appropriate.
- Test persistence after container recreation.
- Troubleshoot authentication and connection failures.
- Never commit database passwords.

Backup and Recovery Flow



Compose Setup (with Backup Volume)

```
services:
  postgres:
    image: postgres:16
    environment:
      POSTGRES_DB: appdb
      POSTGRES_USER: appuser
      POSTGRES_PASSWORD: secret123
    volumes:
      - pgdata:/var/lib/postgresql/data
      - backups:/backups
volumes:
  pgdata:
  backups:
```

1. Create a Backup (pg_dump)

```
docker compose exec postgres \
  pg_dump -U appuser -d appdb \
  -F c -f /backups/appdb.dump
```

- Creates a logical backup in a compressed format (-F c).
- The file is stored inside the container (mounted volume recommended).

2. Restore a Backup (pg_restore)

```
docker compose exec postgres \
  pg_restore -U appuser -d appdb \
  -c /backups/appdb.dump
```

- Restores the database from a pg_dump file.
- The -c option drops existing objects before restore.

3. Test Persistence

1. Start the stack: `docker compose up -d`
2. Create a test table and insert data.
3. Stop and remove the container: `docker compose down`
4. Start again: `docker compose up -d`
5. Verify the data is still there.

Troubleshooting: Common Issues

Issue	Likely Cause	Fix
Authentication failed	Wrong username or password.	Check POSTGRES_USER and POSTGRES_PASSWORD in Compose.
Cannot connect to database	Database not running or wrong host/port.	Run docker compose ps, confirm container is healthy and port 5432 is reachable.
Restore fails	Incorrect file format or permissions.	Use the correct pg_restore flags and ensure the file exists and is readable.

Security Reminder: Never commit database passwords, .env files, or backup files to a Git repository.

Redis Cache + Application

What You'll Learn

- Understand caching and why applications use Redis.
- Deploy Redis with a small application.
- Connect using Compose service discovery.
- Test Redis with redis-cli PING.
- Understand cache hits, misses, and expiration (TTL).

Architecture



docker-compose.yml (Example)

```yaml
services:
  app:
    image: node:20-alpine
    working_dir: /app
    volumes:
      - ./app:/app
    environment:
      - REDIS_HOST=redis
      - REDIS_PORT=6379
    depends_on:
      - redis
    ports:
      - "3000:3000"
  redis:
    image: redis:7-alpine
    ports:
      - "6379:6379"
  networks:
    - app-net
networks:
  app-net:
    driver: bridge
```

Test Redis with redis-cli

```
docker compose up -d                    # start the stack
docker compose exec redis redis-cli     # open Redis CLI
PING                                    # test connection
```

```
root@container:/data# redis-cli
127.0.0.1:6379> PING
PONG
127.0.0.1:6379> SET user:1 "John"
OK
127.0.0.1:6379> GET user:1
"John"
```

Cache Concepts: Hit, Miss, and Expiration

Cache Hit	The data is found in Redis and returned quickly (fast response).
Cache Miss	The data is not in Redis, so the app fetches it from the main source (e.g., database) and may store it in Redis.
Expiration (TTL)	Sets a time-to-live for a key so it is automatically removed after a period. Example: `SET user:1 "John" EX 60 # expires in 60 seconds`

Why Use Redis?

- Faster responses (caches frequently used data).
- Reduces load on your database.
- Improves application performance and scalability.

Junior Engineer Insight:

Use Redis for data that is read often but can be rebuilt or refreshed when needed.

Redis Failures and Cache Recovery

What You'll Learn

- Configure basic health checks for Redis.
- Inspect Redis logs and memory usage.
- Simulate Redis unavailability.
- Understand cache fallback vs. application failure.
- Learn the difference between Redis persistence and eviction policies.

Architecture (with Health Check)



Application (Web Service) ⟷ Redis (with Health Check)

- Health Check
- Logs & Metrics
- Failure Test
- Fallback

docker-compose.yml (Redis with Health Check)

```yaml
services:
  redis:
    image: redis:7-alpine
    ports:
      - "6379:6379"
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]
      interval: 10s
      timeout: 3s
      retries: 5
    volumes:
      - redis-data:/data
volumes:
  redis-data:
```

Inspect Logs and Memory Usage

```
docker compose logs -f redis
# view Redis logs in real time

docker compose exec redis redis-cli
# open Redis CLI

docker compose exec redis redis-cli info memory
# check memory usage
```

Simulate Redis Unavailability

1. Stop Redis: `docker compose stop redis`
2. Test the application (it should fall back).
3. Start Redis again: `docker compose start redis`
4. Check logs and verify recovery.

Cache Fallback vs. Application Failure

Scenario	Result
With Fallback	✓ The app keeps running and uses the database or a default response when Redis is down.
Without Fallback	✗ The app may throw errors or become unavailable if it strictly depends on Redis.

Persistence vs. Eviction Policies

Persistence

- Saves Redis data to disk so it survives a container restart.
- Examples: RDB (snapshot) or AOF (append-only file).

Eviction Policy

- Decides which keys to remove when Redis reaches its memory limit.
- Examples: noeviction, allkeys-lru, volatile-ttl.

Key Commands (Quick Reference)

```
redis-cli ping
# health check (PONG)

redis-cli info memory
# memory usage and stats

redis-cli config get maxmemory-policy
# current eviction policy
```

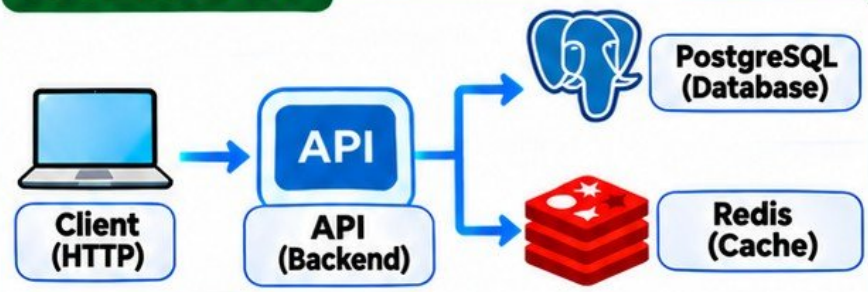
Production Insight: Redis persistence protects data across restarts, while eviction policies control what gets removed when memory is full.

API + PostgreSQL + Redis

What You'll Do

- Build a realistic three-service backend stack.
- Connect the API to its database and cache.
- Use internal service names (api, postgres, redis).
- Understand environment variables and startup dependencies.

Architecture



docker-compose.yaml (Three-Service Stack)

```yaml
services:
  api:
    build: ./api
    ports:
      - "3000:3000"
    environment:
      DATABASE_URL: postgres://appuser:secret123@postgres:5432/appdb
      REDIS_URL: redis://redis:6379
    depends_on:
      - postgres
      - redis
  postgres:
    image: postgres:16
    environment:
      POSTGRES_DB: appdb
      POSTGRES_USER: appuser
      POSTGRES_PASSWORD: secret123
    volumes:
      - pgdata:/var/lib/postgresql/data
  redis:
    image: redis:7-alpine
    volumes:
      - redisdata:/data
volumes:
  pgdata:
  redisdata:
```

Key Configuration

Environment Variables	Pass database and Redis connection details to the API.
Service Names	Use postgres and redis (Compose DNS) instead of container IP addresses.
depends_on	Controls startup order (API waits for PostgreSQL and Redis).

Run and Test the Stack

```
docker compose up -d                    # start all services
docker compose ps                       # check service status
docker compose logs -f api              # view API logs
curl http://localhost:3000/health       # test the API
```

Understand the Connections

PostgreSQL (Database)	■ Stores application data (e.g., users, orders, etc.).
Redis (Cache)	■ Stores frequently used data (e.g., sessions, responses). ■ Makes the API faster and more efficient.

Example API Flow



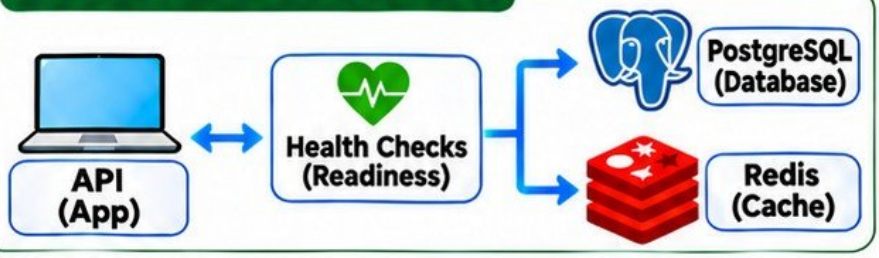
Junior Engineer Insight: Use service names, environment variables, and dependencies to build a clean, maintainable backend stack.

Health Checks and Startup Failures

What You'll Learn

- Configure meaningful service health checks.
- Use depends_on with condition: service_healthy.
- Understand why startup order does not guarantee ongoing availability.
- Troubleshoot database connection refused errors.
- Practice stopping PostgreSQL and observe application behavior.

Architecture and Flow



Health Check Basics

test	The command used to check if the service is healthy.
interval	How often the check runs (e.g., 10s).
timeout	How long to wait for a response (e.g., 5s).
retries	How many failed checks before marking unhealthy (e.g., 5).

docker-compose.yml (Health Checks + Depends_on)

```yaml
services:
  postgres:
    image: postgres:16
    environment:
      POSTGRES_DB: appdb
      POSTGRES_USER: appuser
      POSTGRES_PASSWORD: secret123
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U appuser -d appdb"]
      interval: 10s
      timeout: 5s
      retries: 5
  api:
    build: ./api
    depends_on:
      postgres:
        condition: service_healthy
      redis:
        condition: service_healthy
```

Key Concepts

depends_on	Controls startup order between services.
condition: service_healthy	Starts the service only after the dependency is healthy.
Startup Order ≠ Availability	It only manages startup, not ongoing health (the service can still go down later).

Practice: Simulate a Failure

1. Start the stack: `docker compose up -d`
2. Check health status: `docker compose ps`
3. Stop PostgreSQL: `docker compose stop postgres`
4. Observe API logs: `docker compose logs -f api`
5. Restart PostgreSQL: `docker compose start postgres`

Troubleshoot: Connection Refused

Symptom	Likely Cause	Fix
connection refused	PostgreSQL not running or not ready.	Check service status (docker compose ps), review logs, and add a health check.
timeout connecting	Wrong host, port, or credentials.	Use the service name (e.g., postgres), correct port (5432), and valid environment variables.

Why It Matters

- Health checks give a real "ready" signal.
- The app starts only when its dependencies (PostgreSQL and Redis) are healthy.
- Your app must also handle runtime failures (e.g., the database going down later).

Junior Engineer Insight:

Use health checks for reliable startup and build application-level retries for real resilience.

Prometheus + Grafana Monitoring

What You'll Learn

- Understand metrics collection and dashboards.
- Deploy Prometheus and Grafana using Docker Compose.
- Configure Prometheus scrape targets.
- Connect Grafana to Prometheus.
- Verify target health using the Prometheus interface.

Monitoring Architecture



Key Concepts

Prometheus	Collects and stores time-series metrics (scrapes targets).
Grafana	Visualizes metrics with dashboards, charts, and alerts.
Scrape Target	A service endpoint Prometheus polls (e.g., /metrics) to collect metrics.

docker-compose.yml (Prometheus + Grafana)

```yaml
services:
  prometheus:
    image: prom/prometheus:v2.55.0
    volumes:
      - ./prometheus.yml:/etc/prometheus/prometheus.yml
    ports:
      - "9090:9090"
  grafana:
    image: grafana/grafana:11.2.0
    ports:
      - "3000:3000"
    environment:
      - GF_SECURITY_ADMIN_USER=admin
      - GF_SECURITY_ADMIN_PASSWORD=admin123
```

prometheus.yml (Scrape Targets)

```yaml
global:
  scrape_interval: 15s
scrape_configs:
  - job_name: "api"
    static_configs:
      - targets: ["api:3000"]   # your app metrics

  - job_name: "prometheus"
    static_configs:
      - targets: ["prometheus:9090"]
```

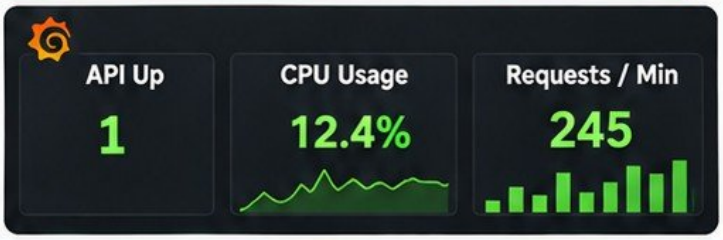
Steps to Deploy and Configure

1. Start the stack: `docker compose up -d`
2. Open Prometheus: http://localhost:9090
3. Check targets: Status → Targets (should be UP)
4. Open Grafana: http://localhost:3000
5. Login (admin / admin123) and add Prometheus as a data source (http://prometheus:9090).
6. Create a dashboard and visualize a metric (e.g., up, process_cpu_seconds_total).

Verify Target Health (Prometheus)



Example Grafana Dashboard (Concept)



Junior Engineer Insight: Prometheus collects the data, and Grafana turns it into meaningful, real-time dashboards for visibility and troubleshooting.

Monitor and Diagnose an Application

What You'll Learn

- Expose application metrics through an endpoint.
- Write a basic PromQL query.
- Create a simple Grafana dashboard.
- Troubleshoot a DOWN scrape target.
- Understand the importance of useful labels and alerts in production.

Monitoring Architecture



Expose Application Metrics (Example: Node.js)

app/metrics.js

```
const client = require('prom-client');
const collectDefaultMetrics = client.collectDefaultMetrics;

collectDefaultMetrics();
app.get('/metrics', async (req, res) => {
  res.set('Content-Type', 'text/plain');
  res.send(await client.register.metrics());
});
```

Sample PromQL Queries

Query	What It Shows
up{job="api"}	Whether the API scrape target is UP (1) or DOWN (0).
rate(http_requests_total {job="api"}[5m])	Request rate (requests per second) for the last 5 minutes.
sum(rate(http_requests_total {job="api"}[5m])) by (status)	Request rate grouped by HTTP status code (e.g., 200, 404, 500).
process_resident_memory_ bytes{job="api"}	Application memory usage (bytes).

Create a Simple Grafana Dashboard

1. Open Grafana: http://localhost:3001
2. Login (e.g., admin / your password).
3. Add data source: Prometheus (URL: http://prometheus:9090).
4. Create a new dashboard → Add panels:
   - Request rate: rate(http_requests_total{job="api"}[5m])
   - HTTP status (200/4xx/5xx): sum(rate(http_requests_total {job="api"}[5m])) by (status)
   - Memory usage: process_resident_memory_bytes{job="api"}

Troubleshoot a DOWN Scrape Target

Possible Cause	Fix
App is not running.	Check container status: docker compose ps
Wrong target URL or port.	Verify the target (e.g., api:3000/metrics) in prometheus.yml.
/metrics endpoint not exposed.	Confirm the app exposes the metrics endpoint and returns 200.
Network or DNS issue.	Ensure services are on the same Docker network (use service name).
App is slow or timing out.	Increase scrape_timeout (e.g., 10s).

Example Grafana Dashboard (Concept)



Production Insight:

- Use meaningful labels (e.g., job="api", env="prod", service="orders").
- Create actionable alerts (e.g., high error rate, high latency, target down).
- Monitoring is not just data collection — it should help you detect, diagnose, and respond to real problems quickly.

Junior Engineer Insight:

Good metrics, clear labels, and the right alerts turn monitoring into faster problem detection and resolution.

RabbitMQ + Background Worker

What You'll Learn

- Understand asynchronous processing with a message queue.
- Deploy RabbitMQ, a producer, and a worker.
- Learn queues, producers, consumers, and acknowledgments.
- Send a sample background job.
- See how the API, queue, and worker work together.

Architecture: API → Queue → Worker



Key Concepts

Queue	A buffer that stores messages (jobs).
Producer	Sends a message to a queue (e.g., the API).
Consumer (Worker)	Reads and processes messages from the queue.
Acknowledgment (ACK)	Confirms the job was processed so it can be removed.

docker-compose.yml (RabbitMQ + API + Worker)

```yaml
services:
  rabbitmq:
    image: rabbitmq:3.13-management
    ports:
      - "5672:5672"    # messaging (AMQP)
      - "15672:15672"  # management UI
  api:
    build: ./api
    environment:
      - AMQP_URL=amqp://guest:guest@rabbitmq:5672
    depends_on:
      - rabbitmq
  worker:
    build: ./worker
    environment:
      - AMQP_URL=amqp://guest:guest@rabbitmq:5672
    depends_on:
      - rabbitmq
```

Sample Job Flow

1. API receives a request (e.g., /send-email).
2. API publishes a job to the queue (email_queue).
3. RabbitMQ stores the job.
4. Worker consumes the job and processes it.
5. Worker acknowledges (ACK) so the job is removed from the queue.

Producer (API) Example

api/producer.js

```js
const amqp = require('amqplib');

async function sendJob() {
  const conn = await amqp.connect(
    process.env.AMQP_URL);
  const ch = await conn.createChannel();
  const queue = 'email_queue';
  await ch.assertQueue(queue, { durable: true });
  const msg = { email: 'user@demo.com' };
  ch.sendToQueue(queue, Buffer.from(
    JSON.stringify(msg)));
  console.log('Job sent:', msg);
  await ch.close();
}
sendJob();
```

Worker (Consumer) Example

worker/worker.js

```js
const amqp = require('amqplib');

async function startWorker() {
  const conn = await amqp.connect(
    process.env.AMQP_URL);
  const ch = await conn.createChannel();
  const queue = 'email_queue';
  await ch.assertQueue(queue, { durable: true });
  ch.consume(queue, (msg) => {
    const job = JSON.parse(msg.content.toString());
    console.log('Processing job:', job);
    // e.g., send email here
    ch.ack(msg); // acknowledge success
  });
}
startWorker();
```

RabbitMQ Management UI

RabbitMQ

Overview | Queues | Exchanges | Connections

Queues

Name	Messages	Ready	Unacked	Total
email_queue	0	0	0	0

Production Insight:

- Use durable queues (survive restarts).
- Acknowledge messages (ACK) to avoid duplicates.
- Handle failures (retry with DLQ or backoff).
- Monitor the queue (e.g., via the management UI or metrics).

Queue Reliability and Worker Failures

What You'll Learn

- Configure durable queues and persistent messages.
- Understand acknowledgments (ACK) and redelivery.
- Simulate a worker crash and observe the outcome.
- Inspect pending messages and consumer state.
- Learn why consumers must handle duplicate deliveries.

Message Reliability Flow



Key Concepts

Durable Queue	Survives RabbitMQ restarts (durable: true).
Persistent Message	Stored on disk (persistent: true).
ACK (Acknowledgment)	Confirms the message was processed.
Redelivery	RabbitMQ re-sends unacknowledged messages.
Idempotency	Handle duplicates safely (same job may run more than once).

Producer: Send a Persistent Message

api/producer.js

```
const amqp = require('amqplib');
async function sendJob() {
  const conn = await amqp.connect(process.env.AMQP_URL);
  const ch = await conn.createChannel();
  await ch.assertQueue('email_queue', { durable: true });
  const msg = { id: 'job-101', email: 'user@demo.com' };
  ch.sendToQueue('email_queue', Buffer.from(JSON.stringify(msg)), {
    persistent: true,          // keep on disk
  });
  console.log('Job published:', msg.id);
  await ch.close(); await conn.close();
}
sendJob();
```

Consumer: Process with ACK and Redelivery

worker/consumer.js

```
const amqp = require('amqplib');
async function startWorker() {
  const conn = await amqp.connect(process.env.AMQP_URL);
  const ch = await conn.createChannel();
  await ch.assertQueue('email_queue', { durable: true });
  ch.prefetch(1);                  // process one message at a time
  ch.consume('email_queue', async (msg) => {
    try {
      const job = JSON.parse(msg.content.toString());
      console.log('Processing job:', job.id);
      // TODO: do the work (e.g., send email)
      ch.ack(msg);                 // acknowledge on success
    } catch (err) {
      console.error('Job failed:', err.message);
      ch.nack(msg,false,true);     // requeue for redelivery
    });
}
startWorker();
```

Simulate a Worker Crash

1. Start the stack: `docker compose up -d`
2. Check queue (should be empty): http://localhost:15672
3. Stop the worker (simulate crash): `docker compose stop worker`
4. Send a few jobs (run the producer).
5. Check the queue again (messages should be pending).
6. Restart the worker: `docker compose start worker` (→ messages are reprocessed).

Inspect the Queue (RabbitMQ UI)

RabbitMQ Management

Overview | **Queues** | Consumers | Connections

Name	Ready	Unacked	Total	Consumers
email_queue	5	0	5	1

Common Outcomes

Scenario	Result
Worker ACKs message	Message is removed from the queue.
Worker crashes (no ACK)	Message is requeued (redelivered later).
Worker fails (nack, true)	Message returns to the queue.
Worker rejects (nack, false)	Message is dropped (or sent to a DLQ).

Production Insight:

- Use durable queues and persistent messages for reliability.
- Design consumers to be idempotent (safe to process a message more than once).
- Consider a Dead Letter Queue (DLQ) for repeated failures (e.g., after several retries).

MinIO + S3-Compatible Object Storage

What You'll Learn

- Understand object storage versus filesystem storage.
- Deploy MinIO using Docker Compose.
- Create a bucket and upload a test file.
- Connect an application through an S3-compatible API.
- Explain endpoints, buckets, objects, and access credentials.

Key Concepts

Endpoint	The API URL used to connect to MinIO (e.g., http://minio:9000).
Bucket	A container that stores objects (e.g., app-files).
Object	A file and its metadata stored in a bucket (e.g., report.pdf).
Access Key	Username (e.g., minioadmin) for API access.
Secret Key	Password (e.g., minioadmin) for authenticating API requests.

Deploy and Test MinIO

1. Start MinIO: `docker compose up -d`
2. Open the MinIO console: http://localhost:9001
3. Login with:
   - Access Key: minioadmin
   - Secret Key: minioadmin
4. Create a bucket (e.g., app-files).
5. Upload a test file (e.g., test.txt).

Verify with AWS CLI (S3 API)

```
# list buckets
aws --endpoint-url http://localhost:9000 \
  s3 ls

# upload a file
aws --endpoint-url http://localhost:9000 \
  s3 cp test.txt s3://app-files/

# list objects
aws --endpoint-url http://looalhost:9000 \
  s3 ls s3://app-files/
```

Object Storage vs Filesystem Storage

Aspect	Filesystem Storage	Object Storage (e.g., MinIO)
Structure	Directories and files (hierarchical).	Flat storage of objects (bucket-based).
Access	Uses local paths (e.g., /data/file.txt).	Uses an API (S3-compatible, HTTP/HTTPS).
Scalability	Limited by server/volume size.	Highly scalable and distributed.
Use Cases	OS files, application data, logs.	Media, backups, artifacts, large unstructured data.

Architecture: Application → MinIO (S3 API)



docker-compose.yml (MinIO)

```
services:
  minio:
    image: minio/minio:latest
    container_name: minio
    ports:
      - "9000:9000"   # S3 API endpoint
      - "9001:9001"   # MinIO web console
    environment:
      - MINIO_ROOT_USER= minioadmin
      - MINIO_ROOT_PASSWORD=minioadmin
    volumes:
      - minio_data:/data
    command: server /data --console-address ":9001"
volumes:
  minio_data:
```

Connect an Application (S3-Compatible API)

app/s3-client.py

```
import boto3

s3 = boto3.client(
    "s3",
    endpoint_url="http://minio:9000",
    aws_access_key_id="minioadmin",
    aws_secret_access_key="minioadmin"
)
# upload a file
s3.upload_file("test.txt", "app-files", "test.txt")
print("File uploaded successfully")
```

Production Insight:

- Use encryption, access policies, and strong credentials.
- Keep backups and enable versioning if needed.
- MinIO is S3-compatible, so applications can use the same S3 SDKs and tooling (e.g., boto3, AWS CLI, or SDKs for Go/Java/Node).

Secure and Troubleshoot Object Storage

What You'll Learn

- Use private buckets and restricted access.
- Configure persistent storage for MinIO.
- Diagnose endpoint and authentication failures.
- Verify file upload and retrieval.
- Understand the production limits of a single MinIO container.

Architecture: Secure Access and Persistence



Key Security Practices

Private Bucket	Not publicly accessible.
Least Privilege	Use a dedicated access key with only required rights.
Access Keys	Use a strong Access Key ID and Secret Access Key.
Persistent Data	Use a named volume (e.g., minio_data) to keep files across restarts.

docker-compose.yml (Persistent MinIO)

```yaml
services:
  minio:
    image: minio/minio:RELEASE.2024-08-17T01-24-54Z
    container_name: minio
    ports:
      - "9000:9000"   # S3 API
      - "9001:9001"   # Web console
    environment:
      MINIO_ROOT_USER: veriqtaadmin
      MINIO_ROOT_PASSWORD: S3StrongPass!2026
    volumes:
      - minio_data:/data   # persistent storage
    command: server /data --console-address ":9001"

volumes:
  minio_data:
```

Configure a Private Bucket (CLI)

```bash
# set MinIO endpoint
export AWS_ENDPOINT_URL="http://localhost:9000"

# create a private bucket
aws --endpoint-url $AWS_ENDPOINT_URL \
   s3 mb s3://app-files

# verify bucket (no public access)
aws --endpoint-url $AWS_ENDPOINT_URL \
  s3 ls
```

Upload and Retrieve a File

```bash
# upload a file
aws --endpoint-url $AWS_ENDPOINT_URL \
  s3 cp test.txt s3://app-files/test.txt
# list objects
aws --endpoint-url $AWS_ENDPOINT_URL \
  s3 ls s3://app-files/
# download the file
aws --endpoint-url $AWS_ENDPOINT_URL \
  s3 cp s3://app-files/test.txt ./downloaded.txt
# verify content
cat downloaded.txt
```

Troubleshoot Common Issues

Issue	Likely Cause	Fix
Could not connect to endpoint	Wrong endpoint URL or port.	Use http://localhost:9000 (S3) or http://localhost:9001 (console).
AccessDenied (403)	Invalid keys or insufficient permission.	Verify Access Key/Secret Key and bucket policy.
No such bucket (404)	Bucket does not exist or wrong name.	Check bucket name and create it (s3 mb s3://app-files).
Upload fails (400)	Wrong file path or request format.	Check the command syntax and file path.

Production Insight

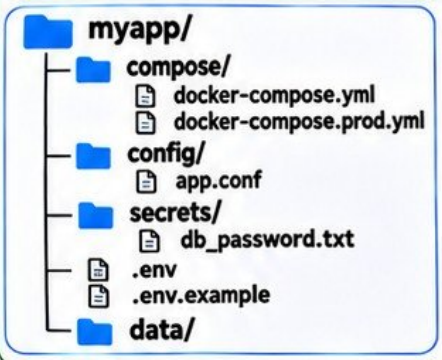
- Keep buckets private by default; avoid public access unless necessary.
- Use persistent storage (named volume) to retain data across container restarts.
- A single MinIO container is not highly available (HA) — it can still be a single point of failure.
- For production, consider a distributed MinIO setup (multiple nodes), backup, and replication.

Make Your Compose Setups Safer and Reusable

What You'll Learn

- Organize Compose files and configuration cleanly.
- Use .env, env_file, and Compose secrets.
- Apply health checks, restart policies, and resource limits.
- Pin image versions and understand image digests.
- Inspect configurations with docker compose config.
- Know when to use Compose vs. an orchestration platform.

Recommended Project Structure



Purpose	
compose/	Compose files (base & prod)
config/	App configuration files
secrets/	Sensitive values (not in Git)
.env	Local environment variables
.env.example	Template for other developers
data/	Persistent data (named volumes)

Environment and Secrets

.env	Default variable file (auto-loaded by Compose).
env_file	Explicitly loads one or more env files in a service.
secrets	More secure for sensitive data (e.g., DB passwords).
Best Practice	Keep .env.example in Git, but never commit real secrets.

Pinning Images

Version Tag	Use a specific, tested version. `image: postgres:16.4`
Image Digest	Pin to an exact image (build) for stronger reproducibility. `image: postgres@sha256:3a9fddde6b3e...`

Useful Commands

Command	Purpose
docker compose config	View the resolved config (without secret values).
docker compose up -d	Start services in the background.
docker compose ps	Check service status.
docker compose logs -f api	Follow service logs.
docker compose down	Stop and remove containers (keep volumes).
docker compose down -v	Stop and remove containers and volumes (⚠ deletes data).

docker-compose.yml (Safer Example)

```yaml
services:
  api:
    image: myapp-api:1.0.0
    env_file:
      - .env
    secrets:
      - db_password
    depends_on:
      db:
        condition: service_healthy
    restart: unless-stopped
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:3000/health"]
      interval: 15s
      timeout: 5s
      retries: 5
    deploy:
      resources:
        limits:
          cpus: "1.0"
          memory: "512M"
        reservations:
          cpus: "0.25"
          memory: "256M"
  db:
    image: postgres:16.4
    environment:
      POSTGRES_USER: ${DB_USER}
      POSTGRES_DB: ${DB_NAME}
    secrets:
      - db_password
    volumes:
      - pg_data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${DB_USER}"]
      interval: 10s
      timeout: 5s
      retries: 5
secrets:
  db_password:
    file: ./secrets/db_password.txt
volumes:
  pg_data:
```

When to Use What?

Docker Compose (Single Host)	Orchestration (e.g., Kubernetes)
✅ Simple, cost-effective setups.	✅ Multiple hosts and higher availability.
✅ Development, testing, and small production workloads.	✅ Automatic scaling and self-healing.
✅ A single server or VM.	✅ Rolling updates and zero-downtime.
✅ Quick to deploy and manage.	✅ More complex, production-grade workloads.

Production Insight:

- Use version control for all Compose and config files.
- Combine health checks, restart policies, and resource limits for resilience.
- Pin image versions (or digests) for reproducible and secure deployments.
- Use Docker Compose for single-host or small production environments; move to orchestration (e.g., Kubernetes) when you need multi-host, high availability, and scaling.

Troubleshoot Seven Real Docker Compose Failures

1 Nginx (502)

Symptom	Likely Cause	Diagnostic Command	Corrective Action	Verification

Nginx returns 502 Bad Gateway.	Upstream API is down, unhealthy, or on the wrong port.	`docker compose ps`
		`docker compose logs api`
		`docker compose logs nginx`
			■ Ensure API is running and healthy.
			■ Fix the upstream URL/port in Nginx config.
				✓ `curl -I http://localhost`
				# Expect: 200 OK

2 PostgreSQL (Data Loss)

Symptom	Likely Cause	Diagnostic Command	Corrective Action	Verification

Database data is missing after container restart.	The named volume was removed (e.g., docker compose down -v).	`docker volume ls`
		`docker inspect <volume>`
		`docker compose logs db`
			■ Restore from a backup (if available).
			■ Recreate the volume and persist data.
			■ Avoid using -v unless intended.
				✓ `docker exec -it db \ psql -U postgres -c "\l"`
				# Expect: database listed

3 Redis (Connection)

Symptom	Likely Cause	Diagnostic Command	Corrective Action	Verification

Redis refuses a connection (e.g., ECONNREFUSED).	Redis container is not running, wrong host/port, or network misconfiguration.	`docker compose ps`
		`docker compose logs redis`
		`docker exec -it redis \ redis-cli ping`
			■ Start or restart Redis container.
			■ Use the correct service name (e.g., redis:6379) in the app.
			■ Verify the containers share the same network.
				✓ `docker exec -it redis \ redis-cli ping`
				# Expect: PONG

4 API (Starts Too Early)

Symptom	Likely Cause	Diagnostic Command	Corrective Action	Verification
API fails at startup with databaee connection errors.	API starts before PostgreSQL is ready to accept connections.	`docker compose logs api` `docker compose logs db`	■ Add a proper DB health check. ■ Use depends_on: condition: service_healthy. ■ Add retry logic in the API (DB connection backoff).	✓ `docker compose ps` # Both services: healthy `curl http://localhost /health` # Expect: 200 OK

5 Prometheus (Cannot Scrape)

Symptom	Likely Cause	Diagnostic Command	Corrective Action	Verification
Prometheus shows target as DOWN and no metrics.	Wrong target URL/ port, /metrics endpoint missing, or network issue.	`docker exec -it prometheus \ cat /etc/prometheus/ prometheus.yml` `curl http://api:3000/metrics`	■ Verify the correct target (e.g., api:3000) in Prometheus config. ■ Ensure the application exposes /metrics. ■ Reload Prometheus after changes: (notice "docker exec -it prometheus \ kill -HUP 1").	✓ Open http://localhost:9090/ targets # Expect: target status UP

6 RabbitMQ (Unprocessed Messages)

Symptom	Likely Cause	Diagnostic Command	Corrective Action	Verification

Messages remain in the queue and are not processed.	Worker is not running, not consuming, or failing before acknowledgment (ACK).	`docker compose ps`
		`docker compose logs worker`
		`docker exec -it rabbitmq \ rabbitmqctl list_queues \ name messages consumers`
			■ Ensure the worker is running and connected to the queue.
			■ Check worker logs for errors and fix them.
			■ Restart the worker if needed.
				✓ `docker exec -it rabbitmq rabbitmqctl \ list_queues`
				# Expect: messages=0 and consumers >= 1

7 MinIO (AccessDenied)

Symptom	Likely Cause	Diagnostic Command	Corrective Action	Verification
MinIO returns AccessDenied (403) when uploading or accessing a file.	Invalid credentials, insufficient permissions, or a bucket policy denying access.	`docker compose logs minio` `aws --endpoint-url \ http://localhost:9000 \ s3 ls`	■ Verify Access/Secret Key environment variables. ■ Ensure the bucket exists and has the correct policy (private or required access). ■ Use the correct endpoint URL.	✓ `aws --endpoint-url \ http://localhost:9000\ s3 ls s3://app-files` # Expect: bucket contents listed

Quick Diagnostic Commands

Command	Purpose
`docker compose ps`	Check container status and health.
`docker compose logs <svc>`	View logs for a specific service.
`docker compose exec <svc> <cmd>`	Run a command inside a container.
`docker volume ls`	List Docker volumes (data persistence).
`docker network ls`	List Docker networks (connectivity).
`docker compose config`	Inspect the final, resolved Compose config.

Production Insight:

- Most Compose issues are caused by incorrect configuration, missing dependencies, or network/port problems.
- Use logs and health checks to find the real cause quickly.
- Keep persistent data in named volumes and back it up regularly.
- Validate service connectivity and endpoints after every change.

Final Challenge: Build a Reusable DevOps Development Stack

Scenario

A development team needs an environment that engineers can start consistently on their own machines.

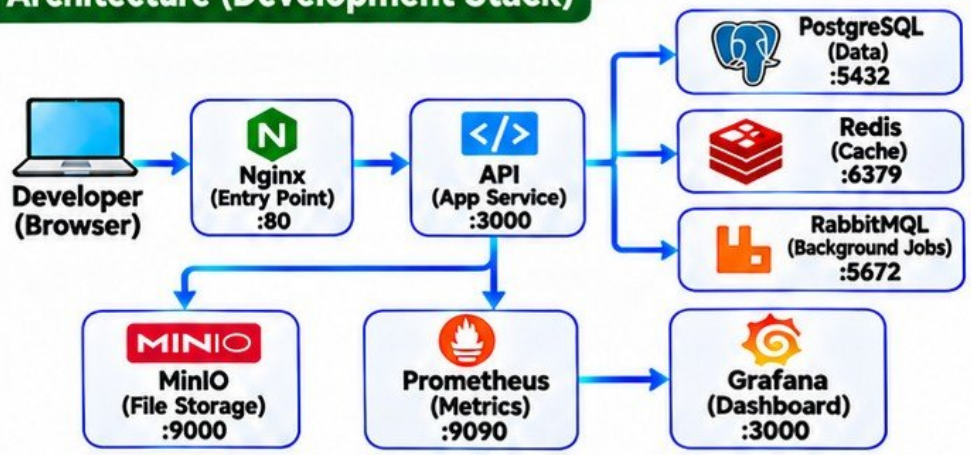
What You'll Build

- Nginx as the entry point.
- A containerized API.
- PostgreSQL for data.
- Redis for caching.
- RabbitMQ for background jobs.
- MinIO for file storage.
- Prometheus and Grafana for monitoring.

Project Structure

```
devops-stack/
├── compose/
│   └── docker-compose.yml
├── nginx/
│   └── default.conf
├── api/
│   └── Dockerfile
├── config/
│   └── prometheus.yml
├── env/
│   └── .env.example
└── data/ (named volumes)
```

Architecture (Development Stack)



docker-compose.yml (Core Services)

```yaml
services:
  nginx:
    image: nginx:1.27-alpine
    ports: ["80:80"]
    volumes: ["./nginx/default.conf:/etc/nginx/conf.d/default.conf:ro"]
    depends_on: [api]
  api:
    build: ./api
    environment:
      - DATABASE_URL=postgresql://app:app@postgres:5432/appdb
      - REDIS_URL=redis://redis:6379
      - RABBITMQ_URL=amqp://app:app@rabbitmq:5672
    depends_on:
      postgres:
        condition: service_healthy
      redis:
        condition: service_started
      rabbitmq:
        condition: service_healthy
  postgres:
    image: postgres:16.4
    environment:
      - POSTGRES_USER=app
      - POSTGRES_PASSWORD=app
      - POSTGRES_DB=appdb
    volumes: ["pg_data:/var/lib/postgresql/data"]
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U app"]
      interval: 10s
  redis:
    image: redis:7.4-alpine
  rabbitmq:
    image: rabbitmq:3.13-management
    ports: ["5672:5672", "15672:15672"]
  minio:
    image: minio/minio:RELEASE.2024-08-17T01-24-54Z
    environment:
      - MINIO_ROOT_USER=miniouser
      - MINIO_ROOT_PASSWORD=MinioPass!2026
    command: server /data --console-address ":9001"
    ports: ["9000:9000", "9001:9001"]
  prometheus:
    image: prom/prometheus:v2.54.1
    volumes: ["./config/prometheus.yml:/etc/prometheus/prometheus.yml"]
    ports: ["9090:9090"]
```

Services and Roles

Service	Role
Nginx	Entry point, reverse proxy (to the API).
API	Application service (e.g., REST API).
PostgreSQL	Persistent relational data (uses a named volume).
Redis	Caching and fast, temporary data (e.g., sessions).
RabbitMQ	Message queue for background jobs.
MinIO	S3-compatible object storage for files.
Prometheus	Collects and stores metrics (from API and services).
Grafana	Visualizes metrics with dashboards and alerts.

Access URLs (Local)

Application (via Nginx):	http://localhost
Grafana Dashboard:	http://localhost:3000 (default: admin/admin)
Prometheus:	http://localhost:9090
RabbitMQ Management:	http://localhost:15672 (default: guest/guest*)
MinIO Console:	http://localhost:9001 (user: miniouser)

Bring Up the Stack

1. Copy the template: `cp env/.env.example .env`
2. Start the stack: `docker compose up -d`
3. Check services: `docker compose ps`
4. View logs (if needed): `docker compose logs -f <service>`

Verify the Environment

- API is reachable: `curl -I http://localhost`
- Database is ready: `docker compose exec postgres psql -U app -d appdb -c "\dt"`
- Redis is working: `docker compose exec redis redis-cli ping`
- RabbitMQ is running: open http://localhost:15672
- MinIO is accessible: open http://localhost:9001
- Prometheus is up: open http://localhost:9090
- Grafana shows metrics: open http://localhost:3000

Production Insight

- Use version-pinned images (e.g., postgres:16.4).
- Keep secrets in a .env file (not in Git).
- Use named volumes for persistent data.
- Add health checks and restart: unless-stopped.
- This stack is ideal for a single machine (dev, testing, and team onboarding).
- Use orchestration (e.g., Kubernetes) when you need multi-host, high availability, and scaling.